

CASOS DE TESTE OBRIGATÓRIOS

Condições de conclusão

Projeto Falcon-8 - Sistema de Controle para Veículos Autônomos

OBJETIVO DESTES DOCUMENTO

Este documento especifica os **casos de teste obrigatórios** que seu processador Falcon-8 **DEVE** executar corretamente. Você deve:

1.  **Implementar** cada caso de teste no RARS (código Assembly)
2.  **Carregar** os valores de entrada na memória do Logisim
3.  **Executar** o programa no seu processador
4.  **Verificar** que as saídas correspondem aos valores esperados
5.  **Demonstrar** o funcionamento no vídeo de apresentação

ATENÇÃO: A professora testará esses mesmos casos durante a avaliação. Seu processador **DEVE** produzir os resultados corretos.

MÓDULO 1: SISTEMA DE FRENAGEM INTELIGENTE (SFI)

Mapa de Memória - Endereços de Dados

Endereço	Nome	Descrição	Unidade	Tipo
0x10	DIST_ATUAL	Distância do obstáculo frontal	metros	Entrada (sensor)

0x11	VEL_ATUAL	Velocidade atual do veículo	km/h	Entrada (sensor)
0x12	DIST_SEGURA	Limite de distância segura (constante)	metros	Entrada (config)
0x13	VEL_MAX_PERMITIDA	Velocidade máxima para a distância	km/h	Entrada (config)
0x20	CMD_FREIO	Comando para o atuador de freio	código	Saída (atuador)
0x21	STATUS_SISTEMA	Status do sistema	código	Saída (display)

Códigos de Comando de Freio (Mem[0x20])

Valor	Significado	Ação do Atuador
0	Freio desativado	Sem ação de frenagem
1	Freio normal	Frenagem suave/moderada
2	Freio de emergência	Frenagem máxima imediata

Códigos de Status (Mem[0x21])

Valor	Significado	Descrição
0	Normal	Sistema operando normalmente
1	Alerta	Situação de atenção (pré-emergência)
2	Emergência	Situação crítica detectada



CASOS DE TESTE DO MÓDULO 1 (SFI)

CASO 1: EMERGÊNCIA - Colisão Iminente

 **Descrição do Cenário:** Pedestre atravessa repentinamente a 2 metros de distância. Veículo trafega a 60 km/h. Distância crítica e velocidade alta exigem frenagem de emergência imediata.

 **VALORES DE ENTRADA (Estado Inicial da Memória):**

```
Mem[0x10] = 2      # Distância atual: 2 metros (CRÍTICO!)
Mem[0x11] = 60     # Velocidade atual: 60 km/h (ALTA!)
Mem[0x12] = 3      # Distância segura: 3 metros (limite)
Mem[0x13] = 30     # Velocidade máxima permitida: 30 km/h (para essa
distância)
```

 **PROCESSAMENTO ESPERADO:**

1. LER valores da memória
2. COMPARAR: distância_atual (2m) < dist_segura (3m)? → SIM ✓
3. COMPARAR: vel_atual (60) > vel_max_permitida (30)? → SIM ✓
4. DECISÃO: Ambas condições verdadeiras → EMERGÊNCIA

 **VALORES DE SAÍDA ESPERADOS:**

```
Mem[0x20] = 2      # Comando: Freio de EMERGÊNCIA
Mem[0x21] = 2      # Status: EMERGÊNCIA
```

 **CRITÉRIO DE APROVAÇÃO:**

- Mem[0x20] deve ser exatamente 2
 - Mem[0x21] deve ser exatamente 2
-

CASO 2: OPERAÇÃO NORMAL - Via Livre

 **Descrição do Cenário:** Veículo trafega em via livre. Sensor detecta obstáculo a 50 metros (muito distante). Velocidade dentro dos limites. Sistema deve permanecer em estado normal.

 **VALORES DE ENTRADA:**

```
Mem[0x10] = 50     # Distância atual: 50 metros (SEGURO)
Mem[0x11] = 40     # Velocidade atual: 40 km/h (MODERADA)
```

```
Mem[0x12] = 3          # Distância segura: 3 metros
Mem[0x13] = 60         # Velocidade máxima permitida: 60 km/h
```

PROCESSAMENTO ESPERADO:

1. LER valores da memória
2. COMPARAR: distância_atual (50m) < dist_segura (3m)? → NÃO ✗
3. DECISÃO: Distância segura → NORMAL

VALORES DE SAÍDA ESPERADOS:

```
Mem[0x20] = 0          # Comando: Freio DESATIVADO
Mem[0x21] = 0          # Status: NORMAL
```

CRITÉRIO DE APROVAÇÃO:

- Mem[0x20] deve ser exatamente 0
 - Mem[0x21] deve ser exatamente 0
-

CASO 3: ALERTA - Distância Crítica, Velocidade OK

 **Descrição do Cenário:** Veículo aproxima-se de um semáforo. Distância está abaixo do limite de segurança, mas velocidade já foi reduzida adequadamente. Sistema deve acionar freio normal (não emergência).

VALORES DE ENTRADA:

```
Mem[0x10] = 2          # Distância atual: 2 metros (ABAIXO do limite)
Mem[0x11] = 20         # Velocidade atual: 20 km/h (BAIXA)
Mem[0x12] = 3          # Distância segura: 3 metros
Mem[0x13] = 30         # Velocidade máxima permitida: 30 km/h
```

PROCESSAMENTO ESPERADO:

1. LER valores da memória
2. COMPARAR: distância_atual (2m) < dist_segura (3m)? → SIM ✓
3. COMPARAR: vel_atual (20) > vel_max_permitida (30)? → NÃO ✗
4. DECISÃO: Distância crítica MAS velocidade adequada → ALERTA (freio normal)

VALORES DE SAÍDA ESPERADOS:

```
Mem[0x20] = 1          # Comando: Freio NORMAL
Mem[0x21] = 1          # Status: ALERTA
```

CRITÉRIO DE APROVAÇÃO:

- Mem[0x20] deve ser exatamente 1
- Mem[0x21] deve ser exatamente 1

CASO 4: LIMITE EXATO - Distância no Limite

 **Descrição do Cenário:** Veículo mantém exatamente a distância de segurança. Velocidade adequada. Sistema deve considerar como operação normal (distância $\geq$ limite).

VALORES DE ENTRADA:

```
Mem[0x10] = 3      # Distância atual: 3 metros (EXATAMENTE no limite)
Mem[0x11] = 30     # Velocidade atual: 30 km/h
Mem[0x12] = 3      # Distância segura: 3 metros
Mem[0x13] = 40     # Velocidade máxima permitida: 40 km/h
```

PROCESSAMENTO ESPERADO:

1. LER valores da memória
2. COMPARAR: distância_atual (3m) < dist_segura (3m)? → NÃO ✗
(igualdade não é "menor que")
3. DECISÃO: Distância no limite = seguro → NORMAL

VALORES DE SAÍDA ESPERADOS:

```
Mem[0x20] = 0      # Comando: Freio DESATIVADO
Mem[0x21] = 0      # Status: NORMAL
```

CRITÉRIO DE APROVAÇÃO:

- Mem[0x20] deve ser exatamente 0
- Mem[0x21] deve ser exatamente 0

CASO 5: ALTA VELOCIDADE - Distância OK, Velocidade Muito Alta

 **Descrição do Cenário:** Veículo trafega em alta velocidade (80 km/h) mas com obstáculo distante (30m). Embora a distância seja segura AGORA, a velocidade alta pode tornar a situação perigosa rapidamente. Sistema deve manter normal pois distância ainda é segura.

VALORES DE ENTRADA:

```
Mem[0x10] = 30     # Distância atual: 30 metros (SEGURO)
Mem[0x11] = 80     # Velocidade atual: 80 km/h (MUITO ALTA)
Mem[0x12] = 3      # Distância segura: 3 metros
Mem[0x13] = 50     # Velocidade máxima permitida: 50 km/h
```

PROCESSAMENTO ESPERADO:

- 1. LER valores da memória
- 2. COMPARAR: distância_atual (30m) < dist_segura (3m)? → NÃO ✗
- 3. DECISÃO: Distância_segura (prioridade) → NORMAL
(Note: velocidade só importa SE distância for crítica)

 VALORES DE SAÍDA ESPERADOS:

Mem[0x20] = 0 # Comando: Freio DESATIVADO
Mem[0x21] = 0 # Status: NORMAL

 CRITÉRIO DE APROVAÇÃO:

- Mem[0x20] deve ser exatamente 0
- Mem[0x21] deve ser exatamente 0

 **OBSERVAÇÃO PEDAGÓGICA:** Este caso testa a lógica condicional: a condição de emergência requer **AMBAS** as situações (distância crítica E velocidade alta). Se apenas uma for verdadeira, não é emergência.

 RESUMO - MATRIZ DE CASOS DE TESTE DO MÓDULO 1

Cas o	Dist. Atual	Vel. Atual	Dist. < Limite?	Vel. > Max?	Cmd Freio	Statu s	Cenário
1	2m	60 km/h	✓ SIM	✓ SIM	2	2	Emergênci a
2	50m	40 km/h	✗ NÃO	✗ NÃO	0	0	Normal
3	2m	20 km/h	✓ SIM	✗ NÃO	1	1	Alerta
4	3m	30 km/h	✗ NÃO	✗ NÃO	0	0	Limite
5	30m	80 km/h	✗ NÃO	✓ SIM	0	0	Vel. Alta

MÓDULO 2: SISTEMA DE VELOCIDADE ADAPTATIVA (CVA) - OPCIONAL

Mapa de Memória - Endereços Adicionais

Endereço	Nome	Descrição	Unidade	Tipo
0x14	VEL_FRENTE	Velocidade do veículo à frente	km/h	Entrada (radar)
0x15	DIST_FRENTE	Distância do veículo à frente	metros	Entrada (radar)
0x22	CMD_ACCELERADOR	Comando PWM para acelerador	0-255	Saída (atuador)
0x23	VEL_ALVO	Velocidade-alvo calculada	km/h	Saída (info)

CASOS DE TESTE DO MÓDULO 2 (CVA) - OPCIONAL

CASO 6: REDUÇÃO DE VELOCIDADE - Veículo Próximo

 **Descrição do Cenário:** Veículo à frente trafega lentamente a apenas 8 metros de distância. Sistema deve reduzir velocidade pela metade para aumentar distância de segurança.

VALORES DE ENTRADA:

Mem[0x10] = 20 # Distância obstáculo frontal: 20m (para SFI)

```
Mem[0x11] = 60      # Velocidade atual: 60 km/h
Mem[0x14] = 40      # Velocidade do veículo à frente: 40 km/h
Mem[0x15] = 8       # Distância do veículo à frente: 8 metros (PRÓXIMO!)
```

PROCESSAMENTO ESPERADO:

1. LER valores da memória
2. VERIFICAR: `dist_frente (8m) < 10m?` → SIM ✓
3. CALCULAR: `vel_alvo = vel_atual >> 1` (shift right = dividir por 2)
`vel_alvo = 60 >> 1 = 30 km/h`
4. CALCULAR: `cmd_acelerador` proporcional à velocidade-alvo
`cmd = (30 * 255) / 120 ≈ 64` (aproximadamente 25% do PWM)

VALORES DE SAÍDA ESPERADOS:

```
Mem[0x22] = 64      # Comando acelerador: ~25% (reduzir)
Mem[0x23] = 30      # Velocidade-alvo: 30 km/h (metade de 60)
```

CRITÉRIO DE APROVAÇÃO:

- Mem[0x23] deve ser aproximadamente **30** (`vel_atual / 2`)
 - Mem[0x22] deve estar no intervalo **[50-80]** (tolerância ± 15 devido a arredondamentos)
-

CASO 7: AUMENTO DE VELOCIDADE - Via Livre Adiante

 **Descrição do Cenário:** Veículo à frente está distante (25 metros). Sistema pode aumentar velocidade para aproveitar via livre, mas respeitando limite máximo.

VALORES DE ENTRADA:

```
Mem[0x10] = 40      # Distância obstáculo frontal: 40m
Mem[0x11] = 50      # Velocidade atual: 50 km/h (PODE ACELERAR)
Mem[0x14] = 70      # Velocidade do veículo à frente: 70 km/h
Mem[0x15] = 25      # Distância do veículo à frente: 25 metros (LONGE!)
```

PROCESSAMENTO ESPERADO:

1. LER valores da memória
2. VERIFICAR: `dist_frente (25m) > 20m?` → SIM ✓
3. CALCULAR: `vel_alvo = vel_atual << 1` (shift left = multiplicar por 2)
`vel_alvo_bruto = 50 << 1 = 100 km/h`
4. LIMITAR: `vel_alvo = min(100, 120) = 100 km/h` (dentro do limite)
5. CALCULAR: `cmd_acelerador = (100 * 255) / 120 ≈ 212`

VALORES DE SAÍDA ESPERADOS:

```
Mem[0x22] = 212     # Comando acelerador: ~83% (acelerar)
```


Mem[0x23] = 100 # Velocidade-alvo: 100 km/h (dobro de 50)

✓ CRITÉRIO DE APROVAÇÃO:

- Mem[0x23] deve ser aproximadamente **100** (vel_atual * 2, limitado a 120)
 - Mem[0x22] deve estar no intervalo **[200-220]** (tolerância ±10)
-

CASO 8: MANUTENÇÃO DE DISTÂNCIA - Zona Intermediária

 **Descrição do Cenário:** Veículo à frente está a distância moderada (15 metros). Sistema deve manter velocidade próxima à do veículo da frente para estabilizar distância.

VALORES DE ENTRADA:

Mem[0x10] = 30 # Distância obstáculo frontal: 30m
Mem[0x11] = 60 # Velocidade atual: 60 km/h
Mem[0x14] = 55 # Velocidade do veículo à frente: 55 km/h (SIMILAR)
Mem[0x15] = 15 # Distância do veículo à frente: 15 metros
(INTERMEDIÁRIO)

PROCESSAMENTO ESPERADO:

1. LER valores da memória
2. VERIFICAR: $10m < \text{dist_frente} (15m) < 20m?$ → SIM (zona intermediária)
3. DECISÃO: vel_alvo = vel_frente (manter distância constante)
vel_alvo = 55 km/h
4. CALCULAR: $\text{cmd_acelerador} = (55 * 255) / 120 \approx 117$

VALORES DE SAÍDA ESPERADOS:

Mem[0x22] = 117 # Comando acelerador: ~46% (ajuste suave)
Mem[0x23] = 55 # Velocidade-alvo: 55 km/h (igual ao da frente)

✓ CRITÉRIO DE APROVAÇÃO:

- Mem[0x23] deve ser aproximadamente **55** (vel_frente)
 - Mem[0x22] deve estar no intervalo **[110-125]** (tolerância ±8)
-

CASO 9: LIMITE MÁXIMO - Controle de Velocidade Superior

 **Descrição do Cenário:** Via livre, veículo poderia acelerar muito, mas sistema deve respeitar limite máximo de 120 km/h por segurança e legislação.

VALORES DE ENTRADA:

Mem[0x10] = 100 # Distância obstáculo frontal: 100m (muito longe)
Mem[0x11] = 80 # Velocidade atual: 80 km/h
Mem[0x14] = 0 # Sem veículo à frente detectado
Mem[0x15] = 100 # Distância: 100m (considera via livre)

PROCESSAMENTO ESPERADO:

1. LER valores da memória
2. VERIFICAR: dist_frente (100m) > 20m? → SIM ✓
3. CALCULAR: vel_alvo_bruto = 80 << 1 = 160 km/h
4. LIMITAR: vel_alvo = min(160, 120) = 120 km/h (LIMITE MÁXIMO)
5. CALCULAR: cmd_acelerador = (120 * 255) / 120 = 255 (máximo)

VALORES DE SAÍDA ESPERADOS:

Mem[0x22] = 255 # Comando acelerador: 100% (máximo permitido)
Mem[0x23] = 120 # Velocidade-alvo: 120 km/h (LIMITE)

CRITÉRIO DE APROVAÇÃO:

- Mem[0x23] deve ser exatamente **120** (respeitando limite máximo)
- Mem[0x22] deve ser exatamente **255** (PWM máximo)



RESUMO - MATRIZ DE CASOS DE TESTE DO MÓDULO 2

Cas o	Vel. Atual	Dist. Frente	Vel. Frente	Condiçã o	Vel. Alvo	Ação
6	60 km/h	8m	40 km/h	< 10m	30	Reduzir (÷2)
7	50 km/h	25m	70 km/h	> 20m	100	Aumentar (×2)
8	60 km/h	15m	55 km/h	10-20m	55	Manter (=frente)
9	80 km/h	100m	0 km/h	> 20m	120	Limite máx



DEMONSTRAÇÃO NO VÍDEO

O que você DEVE mostrar:

1. **Estado Inicial:**
 - Mostrar a memória com os valores de entrada carregados
 - Identificar claramente cada endereço
 2. **Execução:**
 - Executar o programa passo a passo (pode ser em clock manual ou automático)
 - Mostrar o PC avançando
 - Mostrar os valores transitando pelo datapath
 3. **Estado Final:**
 - Mostrar a memória com os valores de saída escritos
 - Comparar com os valores esperados (desta tabela)
 - Explicar por que aqueles valores estão corretos
 4. **Para cada caso de teste:**
 - Tempo sugerido: 2-4 minutos por caso
 - Narrar o que está acontecendo
 - Destacar as decisões do processador
-



CRITÉRIOS DE AVALIAÇÃO

Funcionamento Correto (80% do Módulo):

- ☒ Todos os 5 casos obrigatórios (Módulo 1) funcionam corretamente
- ☒ Valores de saída correspondem EXATAMENTE aos esperados
- ☒ Processador executa sem travamentos

Demonstração (20% do Módulo):

- ☒ Vídeo mostra TODOS os casos de teste
- ☒ Explicação clara do funcionamento
- ☒ Datapath visível e compreensível

Módulo 2 (Opcional - +15 pontos):

- ☒ Mínimo 3 dos 4 casos de teste funcionando (Casos 6, 7, 8)
 - ☒ Demonstração das novas **instruções** (SLL, SRL, BLT)
-



DICAS PARA IMPLEMENTAÇÃO

1. Teste Incremental:

Não implemente tudo de uma vez! Ordem sugerida:

1. Primeiro faça o Caso 2 (Normal - mais simples)
2. Depois o Caso 1 (Emergência - lógica completa)
3. Então os casos 3, 4, 5 (variações)

2. Debugging:

Se algum caso falhar:

- ☒ Verifique os sinais de controle
- ☒ Confira se a **ALU** está fazendo a operação correta
- ☒ Veja se os branches estão sendo tomados corretamente
- ☒ Teste **instruções** individualmente no RARS primeiro

3. Valores Intermediários:

Você pode (e deve!) usar registradores temporários:

- R6, R7 para armazenar resultados de comparações
- R1 para endereços base
- R0 sempre vale 0 (RISC-V padrão)



CHECKLIST FINAL

Antes de submeter seu trabalho, verifique:

- ☐ Todos os 5 casos de teste do Módulo 1 funcionam corretamente
- ☐ Valores de saída conferem com a tabela (Mem[0x20] e Mem[0x21])
- ☐ Vídeo demonstra TODOS os casos de teste obrigatórios
- ☐ Código Assembly está comentado e organizado
- ☐ Datapath no Logisim é claro e bem rotulado
- ☐ (Opcional) Casos 6, 7, 8 do Módulo 2 implementados
- ☐ (Opcional) Pipeline funcionando nos casos de teste